

Lifecycle

Why this exists

InsightFold is a Notebook-Driven Development (NbDD) framework for AFDB and PDBe. The bet is that we validate ideas through lightweight Colab or self-contained notebooks, gather real usage signal, and only graduate what proves useful into production.

For that to work, the team needs a shared understanding of how an idea travels from a one-liner to a stable, integrated feature. This page describes that journey at a high level. Each component has its own child page for the detail.

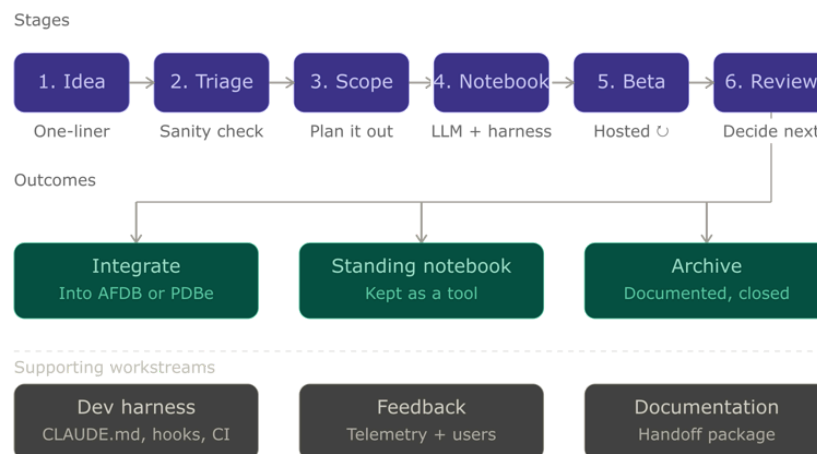
The diagram below summarises the full picture. Each labelled component is something the team can choose to contribute to during the April sprint.

The lifecycle at a glance

Six sequential stages: Idea → Triage → Scope → Notebook prototype → Instrumented beta → Graduation review.

Three possible outcomes from the graduation review: Integrate into AFDB or PDBe, Standing notebook, or Archive.

Three supporting workstreams: Development harness, Feedback, Documentation.



The five stages

1. Idea

A one-liner. What it solves, how, why now. Distilled to its simplest form.

The discipline at this stage is asking the principle question: is the idea simple and novel enough to justify the next step? [ColabFold](#) succeeded because it supported a specific use case extremely well, removing pain points. The same standards apply here.

2. Triage

A healthy exercise before writing any code. A short pitch, supported by enough evidence and reasoning to show the idea is worth exploring. If the pitch is sensible, the ideator has permission to proceed to the notebook stage.

3. Scope

Where an approved idea becomes a concrete specification. A short document that says exactly what the prototype will do, what it will not do, what its inputs and outputs look like, and what "working" means. The discipline is writing this before

any code, so the notebook stage starts with clarity rather than discovering the shape of the prototype while building it.

4. Notebook prototype

The NbDD core. A Colab (or equivalent) notebook, runnable end-to-end on a pinned fixture. The first shippable artefact, not a draft. A colleague should be able to open it, run it, and form an independent view of whether the prototype does what it claims.

5. Instrumented beta

Where a validated notebook becomes a user-facing experience on our own pages, through mirrors of AFDB or PDBe on non-production infrastructure. Users stay inside our surfaces instead of leaving for Colab. Where useful, multiple design variants run side by side so users can pick the one they prefer and return to it with new work.

6. Graduation review

A decision gate. The prototype is evaluated against the criteria set at triage. One of three outcomes is chosen, on a schedule. Not deciding is not allowed, because undecided prototypes go into a state of limbo, which won't help with how .

The three outcomes

Integrate into AFDB or PDBe

Full productionisation. The feature is picked up by the existing AFDB or PDBe engineering teams and integrated into the database proper. SLA, maintenance, long-term ownership.

This outcome sits downstream of the sprint. The engineering teams responsible for integration are not in the sprint. The sprint's contribution is the handoff package that makes integration tractable, not the integration itself.

Standing notebook

The prototype stays alive as a hosted tool, maintained at a lighter touch, without being integrated into the databases. Think ColabFold: never integrated into a database, massively useful anyway.

This outcome matters because conflating it with integration causes teams to either over-invest in productionising every good idea or kill ideas that would thrive as standing notebooks. Keeping them separate is deliberate.

Archive

The prototype is retired. A short retrospective is written, capturing the original idea, what was built, the assumption that failed, and the single condition under which it would be worth revisiting. Code is preserved via Git tag, not actively maintained.

Archive is not a failure outcome. A healthy lifecycle produces archives routinely. The institutional memory this creates prevents the same idea being re-pitched six months later.

The three supporting workstreams

These do not belong to any single stage. They span multiple stages and need deliberate ownership.

Development harness

Active at stages 3 and 4.

The infrastructure that keeps notebook authoring and beta iteration reliable: `CLAUDE.md` conventions, Claude Code hooks, skills, CI, linting, fixture-based testing. The harness is what prevents LLM-assisted development from reaching the end goal while disrespecting everything else that makes code sustainable. It's important we don't limit ourselves to Claude if we can, so that others can adopt the work we create for other LLM and LLM platforms.

Feedback

Active at stages 4 and 5.

The mechanisms that let us listen to users without overwhelming ourselves. We measure with two channels: what people say they prefer, and what they actually return to. A signal is strongest when both agree.

Documentation

Active at stages 2, 3, 4, and 5.

Documentation accumulates across the lifecycle rather than being produced in a final push. By the time a prototype reaches the integration outcome, those accumulated documents are the handoff package that engineering consumes.

Stage-to-concern mapping

Concern	Stages touched
Dev harness	4, 5
Feedback	5, 6
Documentation	2, 3, 4, 5, 6

Stage 1 has no cross-cutting support, which is correct: a one-liner needs no tooling. Stage 2 is only touched by Documentation, which is sufficient for now.

Scope note on integration

Integration into AFDB or PDBe is part of the lifecycle but outside the April sprint. The existing engineering teams for those databases are not in the sprint. The sprint's focus at the back of the lifecycle is making sure that the handoff package produced by the previous stages gives those engineering teams what they need to productionise efficiently when the time comes.

What the sprint can address

The sprint is one week. It cannot address every component above. The purpose of the vote that accompanies this page is to surface where the team wants to contribute, so we can scope the sprint to the components with real energy behind them.